

TABLA DE CONTENIDO

NIVEL FUNDAMENTAL	1
Módulo 3. Pythonic	1
Funciones y Diseños de APIs	1
Lambdas, Closures y Decoradores	3
Iteradores, Generadores y Comprensiones	5
Context Managers	7
Preparación del Laboratorio	8
Estructura del Laboratorio	9

NIVEL FUNDAMENTAL

MÓDULO 3. PYTHONIC

FUNCIONES Y DISEÑOS DE APIS

Funciones. Se definen con *def*, y se ejecutan únicamente escribiendo el nombre. Las funciones permiten agrupar instrucciones reutilizables.

Definición	Ejecución
<pre>def saludar(): print("Hola")</pre>	<pre>saludar()</pre>

Parámetros y argumentos. Un parámetro aparece en la definición de una función, mientras que un argumento es el valor que se envía al llamar la función.

Parámetro	Argumento
<pre>def saludar(nombre): print(f"Hola, {nombre}")</pre>	<pre>saludar("Ana")</pre>

Valor de retorno. Una función puede devolver un resultado mediante *return*, la cual termina su ejecución y entrega el valor al código que la llamó. Si una función *no tiene* *return*, devuelve *None*.

Definición	Ejecución
<pre>def sumar(a, b): return a + b</pre>	<pre>resultado = sumar(3, 5) print(resultado)</pre>

Argumentos posicionales. Se asigna según el orden de los parámetros y argumentos.

Ejemplo	Resultado
<pre>def presentar(nombre, edad): print(f"{nombre} tiene {edad} años")</pre>	"Ana" se asigna a nombre. 30 se asigna a edad.

```
presentar("Ana", 30)
```

Argumentos nombrados. Se especifica el nombre del parámetro. Puede combinarse, pero los posicionales deben aparecer antes. Un argumento nombrado puede **sobrescribir** un valor predeterminado.

Uso	Posicional y Nombrado
<code>presentar(nombre="Ana", edad=30)</code>	<code>presentar("Ana", edad=30)</code>

Valores predeterminados. Parámetros que tienen un valor por defecto. Deben colocarse después de los parámetros obligatorios (posicionales).

Uso	Orden de Uso
<code>def saludar(nombre, mensaje="Hola"):</code> <code> print(f"{mensaje}, {nombre}")</code>	<code>def funcion(obligatorio, opcional="valor"):</code> <code> pass</code>

Argumentos variables '*args'. Permite recibir una cantidad variable de argumentos posicionales.

Definición	Ejecución
<code>def sumar_todos(*numeros):</code> <code> return sum(numeros)</code>	<code>print(sumar_todos(1, 2, 3))</code>

*'numeros' se comporta como **tupla***

Argumentos variables nombrados 'kwargs'.** Permite recibir una cantidad variable de argumentos nombrados. **Importante:** si un argumento nombrado coincide con un parámetro definido, se asigna a ese parámetro y no entra en ****kwargs**.

Definición	Ejecución
<code>def mostrar_datos(**datos):</code> <code> for clave, valor in datos.items():</code> <code> print(f"{clave}: {valor}")</code>	<code>mostrar_datos(nombre="Ana", edad=30, activo=True)</code> <i>'datos' se comporta como diccionario.</i>

Combinar parámetros. El orden que debe respetarse para combinar parámetros en una misma función es:

1) Posicional, 2) Variables, 3) Predeterminados y 4) Variables nombradas.

Definición	Ejecución
<code>def registrar(evento, *etiquetas, prioridad="normal", **metadatos):</code> <code> print(evento)</code> <code> print(etiquetas)</code>	<code>registrar("inicio", "sistema", "produccion", prioridad="alta",)</code>

```
print(prioridad)           usuario="Ana",
print(metadata)           )
```

Diseño de funciones claras. Una función debe tener una responsabilidad clara, para que no hagan demasiadas cosas. Estas suelen: tener un nombre descriptivo, recibir solo datos necesarios, devolver un resultado y ser fácil de probar.

LAMBDAS, CLOSURES Y DECORADORES

Funciones como objetos. En Python, las funciones pueden asignarse a variables y pasarse como argumentos, lo que permite crear funciones que reciben o devuelven otras funciones.

```
def duplicar(numero):
    return numero * 2

operacion = duplicar
print(operacion(5))
```

Funciones Lambda. Una *lambda* es una función pequeña y anónima. Son apropiadas para operaciones simples y breves, pero si la lógica necesitas varias instrucciones o un nombre descriptivo, es preferible usar *def*.

Función Original	Uso con Lambda
<pre>def duplicar(numero): return numero * 2</pre>	<pre>duplicar = lambda numero: numero * 2</pre>
(Código para duplicar valores)	<pre>numeros = [1, 2, 3, 4] dobles = list(map(lambda numero: numero * 2, numeros))</pre>

Closures. Función interna que conserva valores del alcance externo aunque la función externa haya terminado. Se compone de: función externa, función interna, un valor externo que la función conversa.

Definición	Uso
<pre>def crear_multiplicador(factor): def multiplicar(numero): return numero * factor return multiplicar</pre>	<pre>por_dos = crear_multiplicador(2) por_tres = crear_multiplicador(3) print(por_dos(5)) print(por_tres(5))</pre>

En este ejemplo, en **Uso** el primer valor queda guardado como el parámetro *factor*, y para el segundo uso va directo a *numero*.

Decoradores. Recibe una función, agrega comportamiento y devuelve otra función.

Definición	Uso
<pre>def registrar_llamada(funcion): def envoltura(): print("Antes de ejecutar") funcion() print("Después de ejecutar") return envoltura</pre>	<pre>def saludar(): print("Hola") saludar = registrar_llamada(saludar) saludar()</pre>

Sintaxis '@'. Python permite aplicar un decorador con '@'.

Función Original	Uso con @
<pre>def saludar(): print("Hola") saludar = registrar_llamada(saludar)</pre>	<pre>@registrar_llamada def saludar(): print("Hola")</pre>

Decoradores con argumentos variables. Utiliza `*args` y `**kwargs` para que un decorador trabaje con funciones que reciben distintos argumentos. ****args*** recibe los *argumentos posicionales* y los guarda como una *tupla*, mientras que *****kwargs*** recibe *argumentos nombrados* y los guarda como un *diccionario*.

```
def registrar_llamada(funcion):
    def envoltura(*args, **kwargs):
        print(f"Ejecutando {funcion.__name__}")
        resultado = funcion(*args, **kwargs)
        return resultado

    return envoltura

@register_llamada
def sumar(a, b):
    return a + b

resultado = sumar(2, 3)
```

De esta manera, la envoltura (`registrar_llamada`) recibe los argumentos (función `sumar(2, 3)`), ejecuta la envoltura con los números y la regresa.

Decorador `functools.wraps`. Permite conservar el nombre y documentación de la función original, ya que un decorador puede reemplazar la información.

```
from functools import wraps
def registrar_llamada(funcion):
    @wraps(funcion)
    def envoltura(*args, **kwargs):
        print(f"Ejecutando {funcion.__name__}")
        return funcion(*args, **kwargs)

    return envoltura
```

Donde `@wraps(function)` conserva metadatos como:

- `__name__`
- `__doc__`
- Información útil para depuración.

Uso Recomendado para Decoradores.

El decorador debe agregar una responsabilidad clara sin modificar innecesariamente la lógica principal. Son útiles para comportamientos transversales como:

- | | |
|-------------------------|-----------------|
| ✓ Registro de llamadas. | ✓ Autorización. |
| ✓ Medición de tiempo. | ✓ Validaciones. |
| ✓ Reintentos. | ✓ Caché. |

ITERADORES, GENERADORES Y COMPRENSIONES

Iterables e iteradores. Un iterable es un objeto que puede recorrerse como una lista, y un iterador indica el evento actual durante un ciclo.

```
numeros = [1, 2, 3] #iterable
iterador = iter(numeros) #obteniendo iterador
next(iterador) #obtener valores uno a uno
Internamente así funciona un ciclo for.
```

Generadores. Produce valores bajo demanda mediante *yield*.

Definición	Uso
<pre>def contar_hasta(limite): numero = 1 while numero <= limite: yield numero numero += 1</pre>	<pre>for numero in contar_hasta(3): print(numero)</pre>

Similar a un *return*, con la diferencia de que *yield* **pausa** la función y **conserva** su estado para continuar después.

Con <i>return</i>	Con <i>yield</i>
<pre>def crear_lista(): return [1, 2, 3]</pre> <i>Construye y devuelve la lista.</i>	<pre>def crear_generador(): yield 1 yield 2 yield 3</pre> <i>Los valores se producen uno por uno.</i>

Los generadores son útiles para procesar grandes cantidades de datos sin cargar todo en memoria.

Comprensiones. Permite crear una **lista**, diccionario o conjunto de forma concisa, definiendo contenido, rangos o condiciones. Son útiles cuando la transformación es breve y clara, pues si se complica es mejor un bucle o función independiente.

Comprensión de Lista

```
cuadrados = [numero * numero for numero in range(5)]
```

Lista de forma concisa.

```
> [0, 1, 4, 9, 16]
```

```
pares = [numero for numero in range(10) if numero % 2 == 0]
```

Lista con condición.

```
> [0, 2, 4, 6, 8]
```

Comprensión de Diccionario

```
cuadrados = {
    numero: numero * numero
    for numero in range(4)
}
```

```
{
    0: 0,
    1: 1,
    2: 4,
    3: 9,
}
```

Diccionario de forma concisa.

Comprensión de Conjunto

```
letras = {letra.lower()
    for letra in "Python"}
```

Conjunto con valores repetidos eliminados automáticamente.

Expresiones generadoras. Utiliza la misma estructura que las comprensiones, pero utilizan **paréntesis** en lugar de corchetes. Éstas no crean todos los valores inmediatamente. Los objetos que devuelven son **objeto generador**.

Expresiones Generadoras

```
cuadrados = (numero * numero for numero in range(5))
```

Expresión generadora.

```
for cuadrado in cuadrados:
    print(cuadrado)
```

Para recorrer la expresión.

```
lista = list(cuadrados)
```

Para convertirla en lista.

Las listas son mejor para:

- Conservar todos los valores.
- Recorrer varias veces.
- Acceder por índice.

Los generadores se utilizan cuando:

- Los datos son numerosos.
- Solo se necesita procesar sus datos una vez.
- Se desea reducir el uso de memoria.
- Los valores pueden producirse progresivamente.

CONTEXT MANAGERS

Un context manager **controla** la entrada y salida de un **bloque de código**. Se utiliza normalmente con:

```
with recurso:
    # uso del recurso
```

Su objetivo principal es garantizar la limpieza de recursos, incluso cuando ocurre una excepción. Debe encargarse de:

- Preparar un recurso o estado.
- Entregarlo mediante `yield` o `__enter__`.
- Liberarlo o restaurarlo al finalizar.
- Asegurar la limpieza mediante `finally` o `__exit__`.

Con archivos. Al salir del bloque, cierra el archivo automáticamente.

```
with open("datos.txt", encoding="utf-8") as archivo:
    contenido = archivo.read()
```

Protocolo de context manager. Una clase puede implementar:

- ❖ `__enter__`: se ejecuta al entrar al bloque with.
- ❖ `__exit__`: se ejecuta al salir del bloque.

Definición	Uso
<pre>class Recurso: def __enter__(self): print("Recurso abierto") return self def __exit__(self, tipo, valor, traceback): print("Recurso cerrado")</pre>	<pre>with Recurso(): print("Usando recurso") > Recurso abierto Usando recurso Recurso cerrado</pre>

Manejo de excepciones (`__exit__`). Los parámetros de `__exit__` representan información de una **posible excepción**.

- ❖ **tipo**: clase de la excepción.
- ❖ **valor**: instancia de la excepción.
- ❖ **traceback**: información de seguimiento.

Si `__exit__` devuelve **False** o **None**, la excepción continúa propagándose. Si devuelve **True**, la excepción se considera manejada.

Uso de 'contextlib.contextmanager' (@). Crea **context managers** con una función y **yield**. En el ejemplo debajo, el código antes de **yield** prepara el recurso, y el código de **finally** lo libera siempre ejecutando li.

```
from contextlib import contextmanager

@contextmanager
def recurso():
    print("Preparar recurso")

    try:
        yield
    finally:
        print("Liberar recurso")
```

Uso:

```
with recurso():
    print("Procesando")
```

PREPARACIÓN DEL LABORATORIO

- **Context managers para medir tiempo.** Para uso en el laboratorio. En el ejemplo, antes de que ocurra una excepción dentro del bloque, finally calculará e imprimirá la duración.

```
from contextlib import contextmanager
from time import perf_counter

@contextmanager
def medir_tiempo():
    inicio = perf_counter()

    try:
        yield
    finally:
        duracion = perf_counter() - inicio
        print(f"Duración: {duracion:.4f} segundos")
```

Uso:

```
with medir_tiempo():
    sum(range(1_000_000))
```

- **Decorador de reintentos.** El decorador ejecutará una función y volverá a intentarlo si ocurre una excepción esperada. Para ello se usarán:
 - `*args` y `**kwargs` para aceptar cualquier función.
 - `functools.wraps` para conservar sus metadatos.
 - `try-except` para capturar errores.
 - `time.sleep` para esperar entre intentos.

- **Generador por lotes.** El generador recibirá una colección y un tamaño de lote, para lo que se usará `yield`, slicing de listas, validación del tamaño del lote y procesamiento progresivo. El generador **no creará** todos los lotes de una vez.
- **Context manager de temporización.** Medirá cuánto tarda un bloque de código, utilizando `@contextmanager`, `yield`, `perf_counter` y `try-finally`.

```
with medir_tiempo("procesamiento"):
    # operaciones
```

ESTRUCTURA DEL LABORATORIO

Flujo de integración:

```
Crear datos
↓
Procesarlos por lotes
↓
Medir el tiempo total
↓
Simular una operación que puede fallar
↓
Reintentar automáticamente
```

La función `generar_lotes(datos, tamaño)`:

- Valida que el tamaño sea un entero.
- Rechaza valores booleanos.
- Rechaza tamaños menores o iguales a cero.
- Produce lotes progresivamente mediante `yield`.
- Permite que el último lote sea menor que el tamaño solicitado.

Archivo para el context manager de temporización: `'src/temporizador.py'`. Utiliza el context manager `medir_tiempo(nombre)`.

- Utiliza `@contextmanager`.
- Mide la duración con `perf_counter`.
- Ejecuta la limpieza en un bloque `finally`.
- Muestra la duración aunque ocurra una excepción.

Archivo para el decorador de reintentos: `'src/reintentos.py'`. Utiliza el decorador `reintentar(...)`.

- Acepta una cantidad máxima de intentos.
- Permite configurar la espera inicial.
- Implementa backoff progresivo.
- Recibe excepciones específicas para reintentar.
- Conserva los metadatos con `functools.wraps`.

- Relanza la última excepción cuando se agotan los intentos.

Archivo de integración: **'src/main.py'**.

- Crea datos de ejemplo.
- Divide los datos en lotes.
- Procesa cada lote.
- Simula un fallo temporal.
- Reintenta automáticamente la operación.
- Mide el tiempo total del procesamiento.

Donde la salida incluye mensaje de reintento, lotes procesados y duración total.